

Course Project

Phase #3. Optimization

Prof. Jaeseung Choi

Dept. of Computer Science and Engineering

Sogang University

General Information

- **Check "Project #3" in *Assignment* tab of *Cyber Campus***
 - Skeleton code (`Prj3.tgz`) is attached in the post
 - Deadline: **12/24** Wednesday 23:59
 - Submission will be accepted in that post, too
 - **No late submission this time!**
- **Please read the instructions in this slide carefully**
 - The specification of the project is quite complex
 - The slide also contains important submission guidelines
 - If you do not follow the guidelines, **you will get penalty**
 - Also, you must **submit three source code files** this time

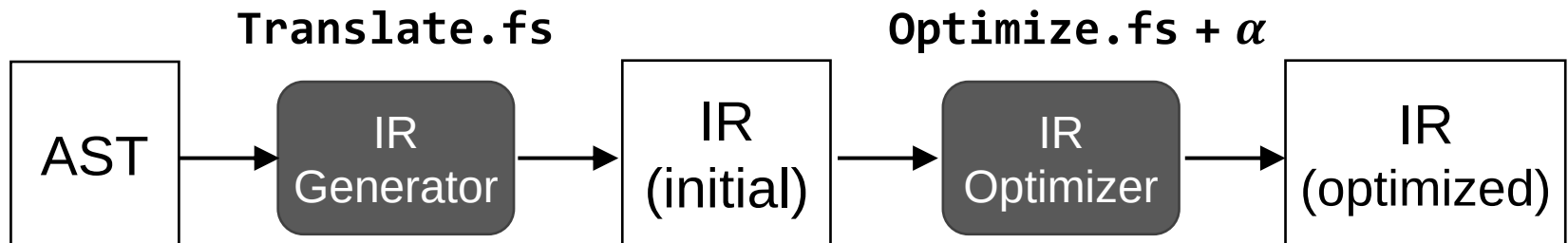
As I said before, this phase will be quite challenging

(Remind) LLM Policy

- **You may get the help from LLMs like ChatGPT**
 - But first try to write the code by yourself
 - Ask LLMs only after that, when you really don't have a clue
- **In this phase, you will realize that LLM is not perfect**
 - To use LLMs wisely, you first have to be a good programmer and have a solid understanding of the relevant CS topics
- **Anyway, you must submit a report if you used LLMs**
 - Write the exact **prompt you entered** to an LLM
 - Also, include the **code you obtained** from that prompt and **how/why you modified** that code
 - If you submit LLM-generated code **without a report**, it will be **considered as cheating**

Outline

- In this phase, you will implement IR-level optimization
 - Make the IR code more efficient, while preserving its behavior
- We use the same language and IR executor to Phase #2
 - But this time, **IR executor will measure and report the execution cost** of your IR code
 - This cost simulates the execution time of the code
- The project structure is similar to Phase #2
 - Some files are added for data-flow analysis and optimization



Skeleton Code (Same as before)

- Copy `Prj3.tgz` into CSPRO server and decompress it
 - You must connect to `csproN.sogang.ac.kr` (N = 2 or 3)
 - Don't decompress-and-copy; copy-and-decompress
- `Optimize.fsproj`: F# project file (you may ignore)
- `src/`: Source files you have to work with
- `testcase/`: Sample test cases and their answers
- `check.py`: Script for self-grading with test cases
- `config`: Used by the grading script (you may ignore)

```
jschoi@cspro2:~$ tar -xzf Prj3.tgz
jschoi@cspro2:~$ ls Prj3/
Optimize.fsproj  check.py  config  src  testcase
```

Structure of src Directory

- Many files have the same role as before
 - **AST.fs**, **Lexer.fs1**, **Parser.fsy**, **Executor.fs** ...
- So I will only explain the updated or new files
 - **Main.fs**: There are four modes supported in this phase
 - **IR.fs**: Execution cost of each instruction is added (**costOf()**)
 - **Translate.fs**: Copy your code from **Phase #2** and improve it
 - **CFG.fs**: Provide modules to construct and use CFG
 - **DFA.fs & Optimize.fs** : You must implement the optimization logic here (they are divided just for better SW design)

```
jschoi@csp2:~/Prj3$ cd src/  
jschoi@csp2:~/Prj3/src$ ls  
AST.fs  CFG.fs  DFA.fs  ...  Optimize.fs  ...  Translate.fs
```

Specification for IR Optimization

- **Correctness:** IR code must behave in the same way before and after the optimization
 - Two functions must return the same output for same input
- **What if input IR code raises an error for certain input?**
 - It's similar to what happened in **Phase #2** (IR translation): your optimized IR code is allowed to do anything for that input
- **In this project, you can simply assume that test case programs do not contain any runtime error**
 - No type mismatch, NULL dereference, division-by-zero, etc.

Optimizations to Implement

- **We will focus on the optimization covered in the lecture**
 - Constant folding
 - Constant propagation
 - Copy propagation
 - Common subexpression elimination
 - Dead-code elimination
 - Mem2Reg (it will be important for performance)
- **Also, fix your `Translate.fs` from Phase #2 to improve the efficiency of the generated IR code**
 - Ex) Think of more efficient translation for short-circuit evaluation

You Mission: `Optimize.run`

- In the `Optimize.fs` file, you have to implement the following function (its type must not change)
 - `let run (ir: IRCode) : IRCode = ...`
 - This function must return the optimized version of IR code
 - I provided a little bit of code as a guideline
- For optimization, you will have to run data-flow analysis
 - You can implement this code in `DFA.fs` (you may choose to put it in `Optimize.fs`, but it will make the file too large)
- CFG module is already implemented for you in `CFG.fs`
 - This will be useful when you implement data-flow analysis
 - Using the functions provided from this module, you can construct a CFG and use it (e.g., find successors of a node)

Four Modes in Main Driver

- In this phase, there are four modes you can run
 - `print-ir` and `run-ir` mode: Same as **Phase #2**
 - **`print-opt`** mode: Print the optimized IR (useful for debugging)
 - **`run-opt`** mode: Run the optimized IR on the provided executor, using the arguments in the input file

```
jschoi@csp2:~/Prj3$ dotnet build -o out
...
jschoi@csp2:~/Prj3$ ./out/Optimize
<Usage>
[*] ./Optimize print-ir <source file>
[*] ./Optimize run-ir <source file> <input file>
[*] ./Optimize print-opt <source file>
[*] ./Optimize run-opt <source file> <input file>
```

Executing the Optimized IR

- When you run the **run-ir** or **run-opt** mode in this phase, it will print out the **execution cost** together
 - By optimizing your IR code to have **less instructions** and use **cheaper instructions**, you can reduce the execution cost

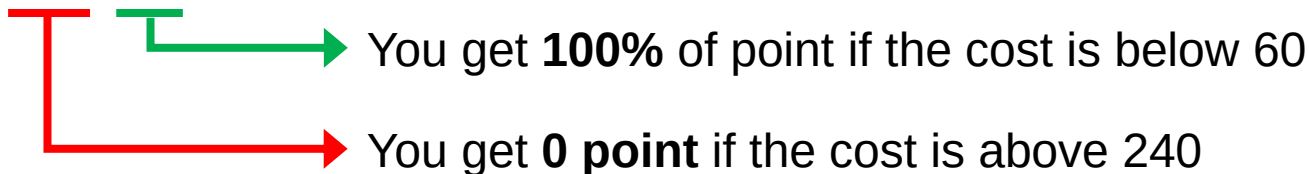
```
jschoi@cspiro2:~/Prj3$ ./out/Optimize run-ir \  
                                testcase/prog-1 testcase/inp-1  
Result: 20, Cost: 88  
Result: 75, Cost: 85  
...  
jschoi@cspiro2:~/Prj3$ ./out/Optimize run-opt \  
                                testcase/prog-1 testcase/inp-1  
Result: 20, Cost: 14  
Result: 75, Cost: 11  
...
```

Evaluation Criteria

- I will use N test programs for grading your optimization, and each program will be executed with M inputs
 - **(Correctness)** If your IR returns wrong output for any of the M inputs, you will get **zero point** for that test program
 - **(Performance)** If your IR returns correct output for all inputs, then your score will be decided by the **sum of execution costs**
- For each test program, I will set two reference points
 - These cost values are stored in `testcase/cost-*`

```
jschoi@cspro2:~/Prj3$ cat testcase/cost-1
```

240 60



Self-Grading

- Once you think everything is done, you can run the **check.py** script as a final check
- Recall that I use different test cases in the real grading
 - So you must think of your own test inputs and check your code with them **(it's especially important for this phase)**
- The checker script will also print out your score based on the evaluation criteria in the previous page
 - You can see **(0/100)** point in the case of wrong output (**X**)

```
jschoi@csp2:~/Prj3$ ./check.py
[*] Result : 0 (70/100) 0 (65/100) 0 (66/100) X (0/100)
```

Submission Guideline

- You should submit three F# source files and report
 - `Translate.fs` (Don't forget this one, too!)
 - `DFA.fs`
 - `Optimize.fs`
 - `report.pdf` (If you used LLM)
- The whole project **must properly compile** when I copy the source code files you submitted
- Submission format
 - Upload these files directly to *Cyber Campus* (do not zip them)
 - **Do not change the file name** (e.g., adding any prefix or suffix)
 - If your submission format is wrong, you will get **-20% penalty**